Especificación de producto · para el equipo de desarrollo

App de Reporte Ciudadano

Gualeguaychú — una app para que el vecino reporte baches, luminarias, basura y otros problemas de la vía pública en cinco pasos, y para que el municipio los reciba, asigne a cuadrilla y resuelva con trazabilidad completa.

11 pantallas principales	**3** modos: vecino, operador e invitado
4 flujos críticos	**3** fases ya cubiertas

Resumen del proyecto `contexto`

La app corre sobre **Expo SDK 54** y habla con la API hermana `api-reporte-ciudadano` (Compose). Del lado del **vecino**, el flujo de reporte es de cinco pasos con foto obligatoria, GPS, texto o voz, adhesión a duplicados cercanos, cola offline en SQLite y push al cambiar el estado. Del lado del **operador**, la bandeja filtra y asigna; el detalle cambia estado, marca duplicado, corrige zona y sube la foto del arreglo; también se puede escanear el QR. El **invitado** mira el mapa público sin cuenta. Las zonas se consultan, no se administran. Abajo: pantallas, componentes, estructura y contrato con la API.

Por qué existe esta app. Hoy el Centro de Atención al Vecino carga todo a mano en una planilla compartida. El trimestre pasado cargaron catorce veces el mismo bache por llamadas repetidas, una rama mal anotada tardó tres semanas en atenderse y cortó la luz de media manzana, y el intendente se enteró de un pozo por Facebook en vez de por el propio equipo. La detección de duplicados a 50 m, el número de seguimiento y los avisos automáticos existen puntualmente para que esas tres cosas no vuelvan a pasar.

> *"El vecino no se enoja porque el bache tarde. Se enoja porque nadie le dice nada."*
>
> — Claudia Benítez, Centro de Atención al Vecino de Gualeguaychú

Sin sesión / invitado — el mapa se mira sin cuenta. El vecino se identifica en `register` (también sirve para reingresar); el operador entra por `login`.

`Pantalla 1` `src/app/(publico)/index.tsx`

Mapa público

- Mapa de la ciudad con los reportes públicos. No muestra quién los hizo
- Filtros por tipo de problema y por estado, y la opción "Cerca mío"
- Resumen: "Hay X reportes cerca tuyo"
- Salidas: Generar reporte (o Identificarme / No soy yo si ya hubo vecino), y "Acceso para operadores municipales"

`Pantalla 2` `src/app/(publico)/register.tsx`

Registro del vecino

- Nombre, email, teléfono y **contraseña** (mínimo 8 caracteres). La API la exige
- Sirve para tres casos: primer reporte (o adhesión) creando la cuenta, reingreso del vecino, y sumarse a un reporte existente
- Sin internet, el primer reporte se encola en el teléfono y se sube al volver la conexión
- Reingreso rápido con rostro o PIN vía `expo-local-authentication`, si el vecino lo eligió

`Pantalla 3` `src/app/(publico)/login.tsx`

Login del operador

- Email y contraseña de la cuenta municipal
- Preferencia de reingreso: rostro, PIN o datos. El token vive en `expo-secure-store` (nunca en el kv-store)
- Al entrar, el layout raíz muestra el grupo `(operador)` y oculta el mapa público

Vecino identificado — tabs: Inicio, Reportar y Mis reportes. El detalle del reclamo es ruta aparte. No hay pantalla de perfil.

`Pantalla 4` `src/app/(tabs)/index.tsx → MapaCiudadano`

Home

- Saludo con el nombre del vecino sobre el mapa de la ciudad
- Mapa con reportes coloreados por estado; filtros por tipo y estado; chip "Cerca mío"
- Resumen "Hay X reportes cerca tuyo" / "Mostramos los reportes de la ciudad"

- Al tocar un pin, tarjeta con acceso al detalle `/reclamo/:id`
- Reportar y Mis reportes viven en la barra de tabs (FAB central → `/reporte/nuevo`)
- Cerrar sesión desde el avatar
- Los avisos push al cambiar el estado los maneja `expo-notifications` + `POST /dispositivos` (si `avisosActivos`)

5. Crear reporte `src/app/reporte/nuevo.tsx`

Una sola pantalla, el asistente de cinco pasos. No hay una ruta por paso. Al enviar, navega al ticket.

Paso 1 — Tipo de problema

Grid de iconos, selección única.

🚧 Bache	💡 Luminaria	🗑️ Basura	🌳 Rama / árbol
💧 Agua / cloaca	🚦 Semáforo	🚶 Vereda	📌 Otro

Paso 2 — Foto (obligatoria)

- Cámara en pantalla completa vía `expo-camera`
- Opción "Elegir de galería"
- Vista previa con opción "Retomar"
- "Agregar segunda foto" (opcional)

Paso 3 — Ubicación

- GPS automático vía `expo-location`
- Pin arrastrable para corregir la posición
- Dirección invertida (geocoding)
- Atajo "Usar ubicación actual". Si niega el permiso, igual puede poner el punto a mano
- Si hay un reporte del mismo tipo a menos de 50 m, pregunta si es el mismo y deja sumarse

Paso 4 — Descripción

- Textarea con placeholder orientativo
- Botón "Grabar nota de voz" vía `expo-audio`
- Timer de grabación en pantalla
- Reproducción de la nota antes de enviar

Paso 5 — Confirmación

- Preview de la foto tomada
- Tipo de problema seleccionado
- Ubicación en un mapa pequeño
- Descripción o nota de voz adjunta
- Zona calculada con el pin (`zonaParaCoordenadas`). No usa la zona del usuario
- Si el pin cae fuera de los cuatro polígonos, queda en Zona Centro
- Botón "Enviar reporte"

`Pantalla 6` `src/app/reporte/enviado.tsx`

Número de seguimiento

- Número de seguimiento visible, con el tipo, el área y la dirección
- Si el vecino se sumó a un reporte existente, el ticket lo dice en vez de mostrar un alta nueva
- Vuelta al inicio

Los cinco estados, con el mismo color en el mapa, la bandeja y el detalle:

🟡 Recibido	🔵 En revisión	🟠 Asignado	🟢 Resuelto	🔴 Rechazado

Pantalla 7 `src/app/(tabs)/mis-reportes.tsx`

Mis reportes

- Pestañas **Propios** y **Sumados** (adhesiones)
- Lista del más nuevo al más viejo; cada ítem muestra estado, dirección y antigüedad
- Los pendientes offline aparecen como "Sin enviar" y se sincronizan al recuperar red o al abrir la pantalla
- Entrar a uno abre `/reclamo/:id`

Pantalla 8 `src/app/reclamo/[id].tsx`

Detalle del reclamo

- Código de seguimiento, por ejemplo GCHU-2026-00412, y el QR para abrirlo en el mostrador sin tipear
- Tipo y dirección
- Estado: recibido, en revisión, asignado a cuadrilla, resuelto o rechazado
- Cuántos vecinos se sumaron
- Foto del problema y, si ya se resolvió, la foto del arreglo
- Línea de tiempo: fecha, estado y comentario. Si está rechazado, el motivo está escrito

Pantallas de operador — requieren autenticación municipal. El vecino no entra acá.

Pantalla 9 `src/app/(operador)/index.tsx`

Bandeja del operador

- Buscador de reportes
- Pestañas: todos, pendientes, mi zona
- Filtros por zona, tipo de problema y estado. Las opciones salen de los servicios
- Por ítem: "Revisar" abre el detalle y "Asignar" abre el sheet de cuadrillas de esa zona
- Escaneo de QR del reclamo (`QRScanner`) para abrir el detalle sin tipear el código
- "Zonas y cuadrillas" abre `/zonas`; Cerrar sesión está en el header
- Cambio de estado, duplicado, foto de arreglo y corrección de zona viven en el detalle

Pantalla 10 `src/app/(operador)/reporte/[id].tsx`

Detalle de gestión

Información visible

- Código, estado, tipo, dirección y zona
- Autor, cantidad de fotos y, si hay cuadrilla, su nombre
- Foto del problema y, si existe, la foto del arreglo
- Si es duplicado, el código del reporte original
- Historial de cambios, con el comentario (incluye eventos de zona corregida)

Acciones, cada una en su sheet

- **Cambiar estado** (`SheetEstado`): comentario obligatorio. Si el estado nuevo es resuelto, la foto del arreglo también
- **Marcar duplicado** (`SheetDuplicado`): elige otro reporte que no sea ya un duplicado
- **Asignar** (`SheetAsignar`): solo cuadrillas activas de la zona del reporte
- **Corregir zona** (`SheetZona`): si el pin cayó en la zona equivocada
- Un reporte resuelto, rechazado o ya marcado como duplicado no se vuelve a gestionar

Pantalla 11 `src/app/(operador)/zonas.tsx`

Zonas y cuadrillas

- Las cuatro zonas se listan siempre. La del operador se marca "Tu zona" y no se ocultan las demás
- Cada tarjeta muestra nombre, referente y "N cuadrillas"

- Debajo, cada cuadrilla con su nombre, especialidad y un chip Activa / Inactiva. El chip se lee, no se toca
- Zona Este, sin cuadrillas: "Todavía no hay cuadrillas en esta zona."
- El operador no da de alta ni de baja zonas ni cuadrillas. El pedido no lo incluye, y Zona no tiene un campo de estado
- La zona del reporte sale del pin al crearlo, no de esta pantalla. El filtro por zona está en la bandeja

Componentes `src/components/`

Agrupados por carpeta. Las pantallas arman la navegación; estas piezas arman el contenido.

`reporte/asistente`
Los cinco pasos en una sola pantalla, más el borrador en curso.

`reporte/paso-*`
Tipo, foto, ubicación, descripción y confirmación. La confirmación muestra la zona del pin.

`reporte/ticket · nota-de-voz`
Constancia con el código; grabación y reproducción de audio con expo-audio.

`operador/sheet-estado`
Cambio de estado con comentario. Pide la foto del arreglo si se resuelve.

`operador/sheet-duplicado · sheet-asignar`
Original del duplicado; cuadrillas activas de la zona del reporte.

`operador/sheet-zona · qr-scanner`
Corregir zona del reclamo; escanear el QR desde la bandeja.

`operador/tarjeta-zona`
Zona, referente, cantidad de cuadrillas y el chip de solo lectura.

`mapa/ · foto/`
Mapa nativo/web y captura a pantalla completa con expo-camera.

`ingreso/ · avisos/`
Desbloqueo (rostro/PIN) para vecino y operador; escucha de push y de red para la cola.

`ui/`
Botón, campo, carga, insignia de estado, icono de categoría y header naranja del operador.

Flujos principales `end-to-end`

Cuatro recorridos que atraviesan varias pantallas y definen la lógica de negocio de la app.

Flujo 1 — Crear reporte (vecino)

1. **Seleccionar tipo de problema**
2. **Tomar foto**
   Obligatoria antes de continuar
3. **Confirmar ubicación**
   Por GPS o ajuste manual del pin
4. **Describir el problema**
   Por texto o nota de voz
5. **Confirmar y enviar**
6. **Mostrar número de seguimiento**
7. **Sin señal**
   Entra a SQLite cola_reportes con sincronizado: false
8. **Recupera señal**
   expo-network dispara el sync; también al abrir Mis reportes

Flujo 2 — Detección de duplicados

1. **El vecino selecciona la ubicación**
2. **El sistema busca reportes a menos de 50 m**

3a. **"¿Es este el mismo problema?" → Sí**
   El vecino se adhiere al reporte existente

"¿Es este el mismo problema?" → No

Continúa con la carga de un reporte nuevo

Flujo 3 — Gestión del operador

1. **Inicia sesión**
2. **Ve la bandeja**
   Pestañas todos, pendientes y mi zona, más filtros
3. **Abre el detalle**
   Fotos, zona, autor e historial
4. **Cambia el estado**
   Con comentario. Si lo resuelve, sube la foto del arreglo
5. **Asigna una cuadrilla activa de esa zona**
   Desde la bandeja o desde el detalle. No ofrece las de otra zona
6. **O lo marca como duplicado / corrige la zona**
7. **El vecino lo ve en el detalle del reclamo**
   Y recibe push si tiene avisosActivos y el dispositivo registrado

Flujo 4 — Seguir el reclamo (vecino)

1. **Entra a Mis reportes**
   Del más nuevo al más viejo
2. **Abre el detalle**
   Estado, foto, adhesiones e historial con comentario
3. **Si está resuelto, ve la foto del arreglo**
4. **Si está rechazado, lee el motivo**
5. **En el mostrador muestra el QR**
   La ventanilla abre el mismo reporte sin tipear el código

Estructura de archivos `Expo Router`

Carpetas reales del proyecto. Las pantallas viven en `src/app` con Expo Router. Los datos no se importan desde las pantallas: pasan por `src/servicios`.

```
src/
├── app/
│   ├── _layout.tsx                # Público, vecino u operador
│   ├── (publico)/
│   │   ├── index.tsx              # Mapa público / invitado
│   │   ├── register.tsx           # Identificación del vecino (+ contraseña)
│   │   └── login.tsx              # Login del operador
│   ├── (tabs)/                    # Vecino identificado
│   │   ├── index.tsx              # Inicio → MapaCiudadano
│   │   ├── reportar.tsx           # Redirige a /reporte/nuevo
│   │   └── mis-reportes.tsx       # Propios, sumados y cola offline
│   ├── reclamo/
│   │   └── [id].tsx               # Detalle del vecino + QR
│   ├── (operador)/
│   │   ├── index.tsx              # Bandeja + escáner QR
│   │   ├── zonas.tsx              # Listado, sin alta ni baja
│   │   └── reporte/
│   │       └── [id].tsx           # Detalle de gestión
│   └── reporte/
│       ├── nuevo.tsx              Asistente de 5 pasos
│       └── enviado.tsx            Ticket
│
├── components/
│   ├── reporte/                   # asistente, pasos, ticket, nota de voz
│   ├── operador/                  # sheets, QR, tarjeta de zona
│   ├── mapa/ · foto/ · avisos/    # mapa, cámara, push/red
│   ├── ingreso/                   # rostro / PIN / preferencia
│   └── ui/
│
├── servicios/                     # api, auth, reportes, bandeja, zonas, push, SQLite…
├── mocks/                         # datos de demo / tests
├── tipos/
└── contexto/                      # sesion + origen-reporte
```

Prioridades de desarrollo `roadmap`

Tres fases de entrega. En el código actual las tres están cubiertas (MVP, core y mejoras: push, voz, QR, adhesiones, foto de arreglo).

Fase 1 — MVP

- ✓ Login y registro
- ✓ Flujo completo de creación de reporte
- ✓ Lista "Mis reportes"
- ✓ Detalle de reporte
- ✓ Persistencia local sin señal

Fase 2 — Core

- ✓ Mapa de la ciudad
- ✓ Detección de duplicados
- ✓ Bandeja de operador
- ✓ Cambios de estado

Fase 3 — Mejoras

- ✓ Notificaciones push
- ✓ Nota de voz
- ✓ Fotos de arreglo
- ✓ Código QR
- ✓ Adhesiones a reportes

Fuera de esta versión `scope`

Cosas que Claudia pidió dejar afuera de la primera entrega. Escritas acá para no sumarlas "de paso" y correr la fecha sin darnos cuenta.

- Turnos, trámites y pagos municipales — esto es solo reclamos
- Chat entre el vecino y el operador — alcanza con los cambios de estado con comentario
- Estadísticas y tableros para las secretarías
- Integración con el sistema de expedientes de la Municipalidad
- Encuesta de satisfacción al cerrar el reclamo
- Alta y baja de zonas o cuadrillas. El operador las consulta; no las crea ni las activa o desactiva

Reglas no negociables `cruzan todas las pantallas`

No son de una pantalla puntual — aplican en toda la app y son las que Claudia remarcó como no negociables.

- Sin foto no hay reporte. Si el vecino no da permiso de cámara, la app se lo explica con buenos modos y le ofrece abrir la configuración del sistema — no lo deja trabado sin salida.
- Si niega la ubicación, tiene que poder reportar igual poniendo el punto a mano en el mapa.
- Buena parte de quienes reclaman tiene más de 60 años: letra grande, botones grandes, y la nota de voz como alternativa real a escribir, no un agregado menor.
- El mapa público muestra el reporte, nunca quién lo hizo.
- Sin señal la app anda igual: el reporte se guarda en el teléfono con `sincronizado: false` y se sube solo al volver la conexión, sin que el vecino tenga que acordarse de nada.
- Crear un reporte tiene que poder hacerse parado en la vereda, en menos de un minuto.

Con qué se construye `stack base`

Qué	Con qué
Framework	`Expo SDK 54 · React Native 0.81 · React 19`
Lenguaje	`TypeScript 5.9`
Navegación	`Expo Router (file-based)`
HTTP	`Axios → api-reporte-ciudadano`

Estilos	`StyleSheet (sin NativeWind / Unistyles)`
Entrega	`EAS · development build (push/FCM no van en Expo Go)`

Capacidades del dispositivo

Capacidad	Paquete	Para qué
Cámara	`expo-camera (CameraView)`	Foto obligatoria y lectura de QR en el mostrador
Galería	`expo-image-picker`	Adjuntar una foto sacada antes
Ubicación y mapa	`expo-location + react-native-maps`	Punto del reclamo, corrección manual, mapa de la ciudad
Notificaciones	`expo-notifications`	Aviso ante cada cambio de estado
Persistencia y red	`expo-sqlite · kv-store · expo-network`	Tablas borradores_reporte y cola_reportes; preferencias no sensibles; sync al volver internet
Seguridad	`expo-secure-store + expo-local-authentication`	Token de sesión; reingreso con rostro o PIN (vecino y operador)
Háptica	`expo-haptics`	Feedback táctil en acciones clave
Multimedia	`expo-audio`	Grabar y reproducir la descripción hablada
QR	`expo-camera + react-native-qrcode-svg`	Mostrar el QR en el detalle y leerlo en la bandeja
Íconos	`@expo/vector-icons`	Ionicons y MaterialCommunityIcons
Gestos	`react-native-gesture-handler · reanimated`	Mapa, sheets e interacción con el pin

Notas de stack que conviene no mezclar con docs viejas:

- `expo-av` ya no — el audio de la nota de voz es `expo-audio`
- Push, SQLite nativo y mapa Android se prueban en *development build*, no en Expo Go
- Los íconos del proyecto son `@expo/vector-icons` (Ionicons / MaterialCommunityIcons)
- Hace falta `google-services.json` + `EXPO_PUBLIC_GOOGLE_MAPS_API_KEY` para FCM y Maps en Android

Modelo de datos `src/tipos/`

La API real ya existe: `api-reporte-ciudadano` (Compose). Los tipos en `src/tipos/` siguen siendo el contrato compartido; la app adapta respuestas Axios en `src/servicios/api.ts`. Los mocks quedan para Jest y datos de apoyo en UI.

Reporte

Propiedad	Tipo	Ejemplo
id	`string`	"rep-00412"
codigo	`string`	"GCHU-2026-00412"
tipoId	`string`	"tip-bache"
descripcion	`string | null`	"Pozo grande, pasa el agua"
audioUrl	`string | null`	nota de voz, si la grabó
fotos	`Foto[]`	al menos una
coordenadas	`Coordenadas`	{ latitud, longitud }
direccion	`string`	"Rocamora 1240"
zonaId	`string`	"zon-norte"
estado	`EstadoReporte`	"asignado"
autorId	`string`	"usr-084"
cuadrillaId	`string | null`	"cua-02"
duplicadoDe	`string | null`	null
fotoArreglo	`Foto | null`	la foto al resolver
adhesiones	`number`	3

creadoEn	`string (ISO)`	"2026-09-14T10:22:00-03:00"
sincronizado	`boolean`	false mientras está en la cola

TipoDeReporte

Propiedad	Tipo
id, nombre, icono, color	`string`
areaResponsable	`string`

CambioDeEstado

Propiedad	Tipo
id, reporteId	`string`
estado	`EstadoReporte`
comentario, operadorId	`string | null`
fechaHora	`string (ISO)`

Usuario

Propiedad	Tipo
id, nombre, email	`string`
telefono	`string | null`
rol	`"vecino" | "operador"`
zonaId	`string | null`
avisosActivos	`boolean`
creadoEn	`string (ISO)`

Zona y Cuadrilla

Propiedad	Tipo
Zona: id, nombre, referente	`string`
Zona: limite	`Coordenadas[]`
Cuadrilla: id, nombre, zonaId, especialidad	`string`
Cuadrilla: activa	`boolean`

Los cinco estados, sin excepción

- recibido
- en_revision
- asignado
- resuelto
- rechazado

Contrato de API y convenciones `api-reporte-ciudadano`

Forma real de las respuestas. La app apunta con `EXPO_PUBLIC_API_URL` / `_EMULADOR` / `_TELEFONO`. En tests (`NODE_ENV=test`) los servicios siguen usando mocks.

```
// éxito
{ "datos": [ ... ], "meta": { "total": 128, "pagina": 1, "porPagina": 20 } }

// error
{ "error": { "codigo": "FOTO_REQUERIDA", "mensaje": "El reporte necesita al menos una foto." } }
```

`ids`

Siempre cadenas de texto, nunca números.

`fechas`

Texto ISO 8601 con zona ("2026-09-14T10:22:00-03:00"). Nunca un objeto Date ni un número.

`claves`

Siempre en camelCase.

`valores ausentes`

Un campo sin valor viene como null — nunca ausente, nunca cadena vacía.

sesión
Authorization: Bearer <token> vía Axios. Token en expo-secure-store, nunca en kv-store.

push
POST /dispositivos con el Expo Push Token; DELETE al cerrar. Avisos al autor si avisosActivos.

fotos API
La API manda fotos con momento problema|arreglo; la app las separa en fotos[] y fotoArreglo.

Estrategia de mocks y servicios `src/mocks/ · src/servicios/`

La API real ya está cableada. Los mocks siguen vivos para Jest y para catálogos de apoyo en UI.

- Los tipos viven en `src/tipos/` y son el contrato compartido con la API
- Los datos de demo/test van en `src/mocks/`, tipados contra esos mismos tipos
- Las pantallas no hablan con Axios directo: pasan por `src/servicios/` (`api.ts`, `auth.ts`, `reportes.ts`, `bandeja.ts`, `zonas.ts`, `cuadrillas.ts`, `notificaciones.ts`, `ubicacion.ts`, `mapa.ts`, `base-datos.ts`…)
- `enPrueba()` hace que en Jest se usen mocks; en la app se llama a la API
- Los mocks de test cubren los casos feos: sin audio, rechazado con motivo largo, lista vacía, varios cambios de estado, error de red / sin conexión